stockflow-erp-api

Modèle de données — document de conception

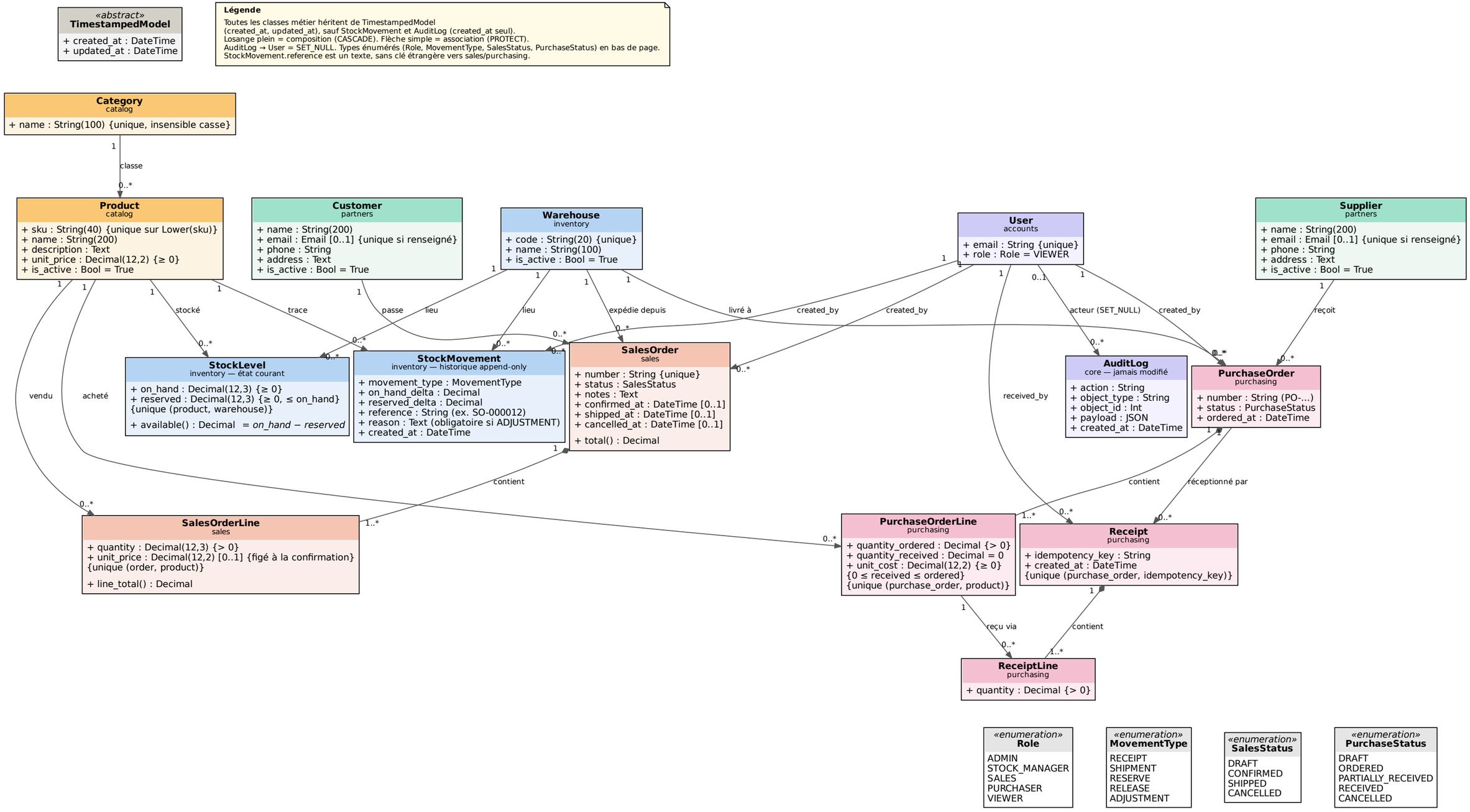
Diagramme de classes final, règles de gestion, concurrence et invariants

Contenu de ce document

Page 2	Diagramme de classes complet (format paysage, vectoriel)
Page 3	Comment lire le diagramme et vue d'ensemble
Page 4	Choix techniques transversaux et stratégies de suppression
Pages 5–7	Description détaillée des modèles, champs et contraintes
Page 8	Cycles de vie et effets sur le stock
Pages 9–10	Transactions, verrous et concurrence
Page 11	Invariants à tester

Le projet en une phrase. stockflow-erp-api est une API de gestion de stock et de flux commerciaux : on achète à des fournisseurs, on stocke dans des entrepôts, on vend à des clients, et chaque variation de quantité est tracée et protégée contre les erreurs de concurrence.

stockflow-erp-api — Diagramme de classes (version finale)



Lire le diagramme de classes

Le diagramme de la page 2 décrit les **15 classes métier**, la classe abstraite `TimestampedModel` et **4 énumérations**. Chaque classe est un modèle Django, qui devient une table PostgreSQL.

Les couleurs

Couleur	Application Django	Classes
Violet	`accounts`, `core`	User, AuditLog
Vert	`partners`	Customer, Supplier
Orange	`catalog`	Category, Product
Bleu	`inventory`	Warehouse, StockLevel, StockMovement
Saumon	`sales`	SalesOrder, SalesOrderLine
Rose	`purchasing`	PurchaseOrder, PurchaseOrderLine, Receipt, ReceiptLine
Gris	`core` (abstrait)	TimestampedModel + les 4 énumérations

Les symboles

Symbole	Signification	Effet en base
Flèche simple	Association : une clé étrangère (FK) de la classe en pointe de flèche vers l'autre côté	`on_delete=PROTECT`
Losange plein	Composition : la ligne appartient à sa commande et disparaît avec elle	`on_delete=CASCADE`
`1`, `0..*`, `1..*`	Cardinalités : exactement un, zéro ou plusieurs, un ou plusieurs	Obligatoire ou facultatif
`{…}`	Contrainte portée par le champ ou la classe	`UNIQUE`, `CHECK`
`+ nom()`	Propriété calculée, jamais stockée	Aucune colonne

Les trois grands blocs

- **Référentiel** : `Category`, `Product`, `Warehouse`, `Customer`, `Supplier`. Ce sont les données qui changent peu.
- **Stock** : `StockLevel` donne l'état courant, `StockMovement` garde l'historique. Les deux doivent toujours rester cohérents.
- **Flux** : ventes (`SalesOrder`) et achats (`PurchaseOrder`, `Receipt`) qui déclenchent les mouvements de stock.

> **Deux choix de conception à retenir.** (1) `StockMovement.reference` est un simple texte (`SO-000012`), pas une clé étrangère : l'application `inventory` ne dépend donc pas de `sales`. (2) `AuditLog → User` est en `SET_NULL` : l'audit survit à la suppression d'un utilisateur.

Choix techniques et suppression

Choix transversaux

Sujet	Décision	Pourquoi
Argent	`DecimalField(12, 2)`	Le type `float` fait des erreurs d'arrondi (0.1 + 0.2 ≠ 0.3)
Quantités	`DecimalField(12, 3)`	Autorise les kilos et les mètres ; l'entier reste possible plus tard
Suppression	`on_delete=PROTECT` partout où une donnée métier existe	Empêche de casser l'historique (règle R11)
Désactivation	`is_active` sur partenaires, produits, entrepôts	« Supprimer » veut dire désactiver
Horodatage	`created_at`, `updated_at` via `TimestampedModel` (dans `core`)	Traçabilité, sans répéter le code
Clés	`BigAutoField` par défaut	Simple

Stratégies de suppression

Relation	`on_delete`	Raison
Ligne → commande (vente, achat, réception)	`CASCADE`	La ligne n'a pas de sens sans sa commande
Commande → client, fournisseur, entrepôt	`PROTECT`	Conserver l'historique
Ligne ou mouvement → produit	`PROTECT`	Conserver l'historique
Produit → catégorie	`PROTECT`	On ne supprime pas une catégorie utilisée
AuditLog → acteur	`SET_NULL`	L'audit survit à l'utilisateur

Ce que cela change au quotidien

- Un produit déjà vendu ne peut plus être supprimé : on le désactive, et il disparaît des nouvelles commandes.
- Supprimer une commande brouillon supprime ses lignes automatiquement (`CASCADE`).
- Si un utilisateur quitte l'entreprise, ses actions restent dans l'audit, avec un acteur vide.

> **Point d'attention : le modèle utilisateur.** `accounts.User` hérite de `AbstractUser` et doit être défini **dès la première migration**. Changer de modèle utilisateur après coup est très pénible. `USERNAME_FIELD = "email"` ; le champ `username` est conservé pour rester simple, ou supprimé (décision du jour 4).

Les modèles en détail (1/3)

accounts.User

Champ	Type	Contraintes
email	EmailField	unique, obligatoire
role	CharField(choices)	ADMIN, STOCK_MANAGER, SALES, PURCHASER, VIEWER ; défaut VIEWER

partners.Customer et partners.Supplier

Champ	Type	Contraintes
name	Char(200)	obligatoire
email	Email	facultatif ; unique si renseigné (index unique partiel)
phone, address	Char, Text	facultatif
is_active	Bool	défaut True

catalog.Category et catalog.Product

Champ	Type	Contraintes
Category.name	Char(100)	unique, insensible à la casse
Product.sku	Char(40)	unique sur `Lower(sku)` (R10)
Product.name	Char(200)	obligatoire
Product.description	Text	facultatif
Product.category	FK Category	PROTECT, obligatoire
Product.unit_price	Decimal(12,2)	CHECK ≥ 0
Product.is_active	Bool	défaut True

```
class Meta:
    constraints = [
        models.UniqueConstraint(Lower("sku"), name="uniq_product_sku_ci"),
        models.CheckConstraint(condition=models.Q(unit_price__gte=0),
                               name="product_price_gte_0"),
    ]
```

Depuis Django 5.1, le paramètre s'appelle `condition` (l'ancien check est déprécié).

inventory.Warehouse

Champ	Type	Contraintes
code	Char(20)	unique
name	Char(100)	obligatoire
is_active	Bool	défaut True

Les modèles en détail (2/3)

inventory.StockLevel : l'état courant

Champ	Type	Contraintes
product, warehouse	FK	PROTECT ; `UniqueConstraint(product, warehouse)`
on_hand	Decimal(12,3)	CHECK ≥ 0 (R1)
reserved	Decimal(12,3)	CHECK ≥ 0 et CHECK reserved ≤ on_hand (R1, R2)

`available = on_hand − reserved` est une propriété calculée, jamais stockée : pas de duplication, donc pas d'incohérence possible.

inventory.StockMovement : l'historique, append-only

Champ	Type	Notes
product, warehouse	FK PROTECT	
movement_type	Char	RECEIPT, SHIPMENT, RESERVE, RELEASE, ADJUSTMENT
on_hand_delta	Decimal	positif ou négatif
reserved_delta	Decimal	positif ou négatif
reference	Char	ex. SO-000012, PO-000004 (texte, pas de FK vers sales)
reason	Text	obligatoire pour ADJUSTMENT
created_by	FK User	PROTECT
created_at	DateTime	auto

Immuable (R9) : l'API n'expose que GET, et `save()` refuse la modification d'une ligne existante. **Cohérence vérifiable** : `sum(on_hand_delta) == StockLevel.on_hand` pour chaque (produit, entrepôt).

sales.SalesOrder et SalesOrderLine

Champ	Type	Notes
SalesOrder.number	Char	unique, ex. SO-000012, généré par le service
customer, warehouse	FK PROTECT	client actif obligatoire ; entrepôt d'expédition
status	Char	DRAFT, CONFIRMED, SHIPPED, CANCELLED
confirmed_at, shipped_at, cancelled_at	DateTime null	dates de transition
Line.order	FK	CASCADE ; `Unique(order, product)`
Line.quantity	Decimal(12,3)	CHECK > 0 (R14)
Line.unit_price	Decimal(12,2) null	null en DRAFT, figé à la confirmation (R6)

`line_total = quantity × unit_price` et `order.total` (somme des lignes) sont calculés, sans colonne stockée au départ.

Les modèles en détail (3/3)

purchasing : PurchaseOrder, PurchaseOrderLine, Receipt, ReceiptLine

Modèle	Champs clés	Contraintes
PurchaseOrder	number (PO-…), supplier, warehouse (destination), status, ordered_at, created_by	FK en PROTECT ; statuts DRAFT, ORDERED, PARTIALLY_RECEIVED, RECEIVED, CANCELLED
PurchaseOrderLine	purchase_order, product, quantity_ordered, quantity_received (défaut 0), unit_cost	ordered > 0 ; 0 ≤ received ≤ ordered (R12) ; `Unique(purchase_order, product)`
Receipt	purchase_order, idempotency_key, received_by, created_at	`Unique(purchase_order, idempotency_key)` (R13)
ReceiptLine	receipt, po_line, quantity	receipt en CASCADE, po_line en PROTECT ; quantity > 0

core.AuditLog

`actor` (FK User, SET_NULL), `action` (ex. `sales_order.confirmed`), `object_type`, `object_id`, `payload` (JSONField), `created_at`. Jamais modifié.

Cycles de vie et effets sur le stock

Les statuts d'une commande suivent un chemin précis. Toute transition en dehors de ce chemin est refusée (`InvalidTransition`), ce qui rend les actions idempotentes.

Commande de vente

Transition	Effet attendu sur le stock	Mouvement créé
DRAFT → CONFIRMED	Réserve la quantité : `reserved` augmente ; le prix de chaque ligne est figé	RESERVE
CONFIRMED → SHIPPED	La marchandise sort : `on_hand` et `reserved` diminuent	SHIPMENT
CONFIRMED → CANCELLED	Libère la réservation : `reserved` diminue (R7)	RELEASE
DRAFT → CANCELLED	Aucun effet : rien n'avait été réservé	aucun

Commande d'achat

Statut	Signification
DRAFT	Commande en préparation, modifiable
ORDERED	Envoyée au fournisseur, aucune réception
PARTIALLY_RECEIVED	Au moins une réception, mais certaines lignes ne sont pas complètes
RECEIVED	Toutes les lignes sont reçues (received = ordered)
CANCELLED	Annulée

Chaque réception augmente `on_hand` (mouvement **RECEIPT**) et `quantity_received` de la ligne concernée. Les corrections manuelles passent par un mouvement **ADJUSTMENT**, dont le champ `reason` est obligatoire.

Exemple chiffré

Étape	on_hand	reserved	available	Mouvement
État initial	10	0	10	
Commande A de 4 confirmée	10	4	6	RESERVE +4 (reserved)
Commande A expédiée	6	0	6	SHIPMENT −4 (on_hand), −4 (reserved)
Commande B de 8 confirmée	—	—	—	Refusée : 8 > 6 disponibles (InsufficientStock)
Réception de 5	11	0	11	RECEIPT +5 (on_hand)

Contrôle : la somme des `on_hand_delta` vaut 10 + (−4) + 5 = 11, soit exactement `on_hand`. C'est l'invariant testé en fin de document.

Remarque. Les effets de chaque transition sur `on_hand` et `reserved` sont la lecture logique des types de mouvement et des invariants du modèle. Les règles exactes (R6, R7…) restent celles de votre cahier des charges.

Transactions, verrous et concurrence (1/2)

Le problème

Deux commerciaux confirment simultanément deux commandes de 8 unités alors qu'il en reste 10.

Temps	Commercial A	Commercial B	État
1	Lit available = 10 → OK		reserved = 0
2		Lit available = 10 → OK	reserved = 0
3	Réserve 8	Réserve 8	**reserved = 16 > on_hand = 10** : incohérence

Une vérification Python seule (`if available >= qty`) ne suffit pas : entre la lecture et l'écriture, une autre requête peut passer. C'est une *race condition*.

Les trois défenses, cumulées

Défense	Rôle
`transaction.atomic()`	Tout ou rien : si une exception est levée, aucune réservation partielle (R4)
`select_for_update()`	Verrou de ligne : B attend que A termine, puis relit la valeur à jour (2 disponibles) et échoue proprement
Contraintes SQL `CHECK`	Filet de sécurité ultime : si un bug contourne le code, PostgreSQL refuse l'écriture (`IntegrityError`)

Squelette du service de confirmation (jour 8)

```
from django.db import transaction

@transaction.atomic
def confirm_sales_order(order_id, user):
    order = SalesOrder.objects.select_for_update().get(pk=order_id)
    if order.status != SalesOrder.Status.DRAFT:
        raise InvalidTransition(...)
    lines = list(order.lines.select_related("product").order_by("product_id"))
    if not lines:
        raise BusinessRuleError("empty_order")
    for line in lines:            # ordre constant -> évite les deadlocks
        level = (StockLevel.objects.select_for_update()
                 .filter(product=line.product, warehouse=order.warehouse).first())
        if level is None or level.available < line.quantity:
            raise InsufficientStock(...)
        level.reserved += line.quantity
        level.save(update_fields=["reserved"])
        StockMovement.objects.create(...)    # RESERVE
        line.unit_price = line.product.unit_price
        line.save(update_fields=["unit_price"])
    order.status = SalesOrder.Status.CONFIRMED
    ...
```

Ce squelette illustre le raisonnement, ce n'est pas du code final exécutable. Dans le document d'origine, l'appel `get_or_none` n'existe pas dans Django : on écrit à la place un `.filter(...).first()`, comme ci-dessus. `select_for_update()` ne fonctionne qu'à l'intérieur d'une transaction.

Transactions, verrous et concurrence (2/2)

Le deadlock

Si A verrouille les produits 1 puis 2, et B les produits 2 puis 1, ils s'attendent mutuellement : c'est un *deadlock*. La parade est de **toujours verrouiller dans le même ordre**, ici `order_by("product_id")`.

Sans ordre constant	Avec ordre constant
A : verrouille 1, attend 2 B : verrouille 2, attend 1 **Blocage mutuel**	A : verrouille 1, puis 2 B : attend 1 (déjà pris par A) **B passe après A, sans blocage**

Réception idempotente

Le client envoie un `idempotency_key` (UUID). Si le réseau coupe et que le client rejoue la requête, la contrainte `Unique(purchase_order, idempotency_key)` (R13) détecte le doublon : le service renvoie la réception existante au lieu de recréer du stock.

Tentative	Clé envoyée	Résultat
1re requête	`a1b2…`	Réception créée, stock +5
Rejeu après coupure réseau	`a1b2…`	Doublon détecté, réception existante renvoyée, stock inchangé
Nouvelle réception volontaire	`c3d4…`	Nouvelle réception créée

Création concurrente d'un StockLevel

Deux réceptions simultanées d'un produit jamais stocké peuvent tenter de créer deux lignes. La contrainte `Unique(product, warehouse)` fait échouer la seconde ; le service rattrape `IntegrityError` et relit la ligne avec verrou.

Résumé des protections

Risque	Protection	Où
Stock réservé en double	`select_for_update()` + `CHECK`	Service + base
Réservation partielle après erreur	`transaction.atomic()`	Service
Deadlock entre deux commandes	Verrouillage trié par `product_id`	Service
Réception rejouée	`Unique(purchase_order, idempotency_key)`	Base
Deux StockLevel pour le même couple	`Unique(product, warehouse)` + rattrapage	Base + service
Historique modifié	API en GET seul + `save()` bloqué	API + modèle

Invariants à tester

Un invariant est une propriété qui doit rester vraie quoi qu'il arrive. Ces tests sont le meilleur filet pour détecter une régression sur la logique de stock.

Invariant	Test
`on_hand ≥ 0`, `reserved ≥ 0`, `reserved ≤ on_hand`	Contraintes en base : vérifier qu'une `IntegrityError` est levée
Σ `on_hand_delta` = `on_hand`	Après une séquence de ventes et de réceptions
Σ `reserved_delta` = `reserved`	Idem
Confirmer deux fois ne réserve qu'une fois	Test d'idempotence de transition
Annuler après confirmation restaure `reserved`	Test de la règle R7

Tests complémentaires recommandés

- Rejouer une réception avec la même `idempotency_key` : le stock ne bouge pas.
- Deux confirmations concurrentes sur le dernier stock disponible : une seule réussit.
- Tenter de modifier un `StockMovement` existant : l'opération est refusée.
- Créer deux produits avec des SKU différant seulement par la casse : le second est refusé.
- Recevoir plus que la quantité commandée : refusé par `quantity_received ≤ quantity_ordered`.
- Supprimer un produit ayant un historique : refusé (`PROTECT`).

> **Prochaine étape :** 06 — API. Les modèles ci-dessus deviennent des endpoints REST ; les mouvements de stock n'exposent que `GET`, et les actions métier (confirmer, expédier, annuler, réceptionner) passent par des services transactionnels.